

Entorno de trabajo. Instalación y primer programa con Android Studio.

Caso práctico



[Mikhail](#) ([Pexels](#))

Juan y María conocen lo fundamental de los dispositivos móviles (las peculiaridades de su hardware, sus sistemas operativos) sobre los que sus aplicaciones van a ejecutarse, y juntos han tomado la decisión de trabajar con **Android Studio**, el lenguaje **Java** y los archivos **XML**.

- María, ¿comenzamos a descargar el software donde desarrollaremos las aplicaciones? - pregunta Juan.

- Sí -responde María-. Tengo ganas de comenzar a programar.

Ada, que les está escuchando, sabe por su experiencia que todavía les quedan cosas por hacer antes de comenzar la instalación.

- Chicos, veo que tenéis muchas ganas pero antes de comenzar a programar e instalar el **IDE** que hemos seleccionado, es imprescindible consultar los requisitos del software -les indica Ada-. Además también hay que revisar las recomendaciones de instalación del fabricante del IDE.

- De acuerdo -aceptan María y Juan al unísono-. Comenzaremos revisando los requisitos y recomendaciones para la instalación del software. ¡Muchas gracias por el consejo!



[Ministerio de Educación y Formación Profesional](#). (Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

[Aviso Legal](#) 

1.- Módulos para el desarrollo de aplicaciones. Instalación de Android Studio

Caso práctico



[Mikhail](#) ([Pexels](#))

Juan, por iniciativa propia, ya ha avanzado en este trabajo previo a la instalación de cualquier software de uso profesional. De hecho, a partir de la información que Ada le ha transmitido en estas horas de trabajo previas al desarrollo, ya ha navegado por varias páginas de la web developer.android.com .

-Yo tengo la **descripción general** de la herramienta y sus componentes, una **guía de instalación** y el **enlace para su descarga** con los requisitos de hardware necesarios -indica Juan-.

-¡Qué buen trabajo has realizado, Juan! -le felicita María.

-He comprobado que los requisitos son bastante altos -añade Juan-, ya que el **IDE** incluye muchas herramientas que facilitan nuestro trabajo.

-¿Es necesario instalar todas? -pregunta Pedro.

-No, en realidad no serían todas imprescindibles -responde Juan-, pero nuestro trabajo será más productivo conociéndolas e instalándolas.

De este modo, Juan ha compartido todos estos conocimientos con el equipo de trabajo, al que se ha unido Pedro. Pedro acaba de terminar otro proyecto de desarrollo web, y el cliente le ha pedido que parte de la funcionalidad desarrollada en la página web pueda incorporarse a una aplicación para dispositivos móviles.

Módulos para desarrollar una aplicación Android

Una vez realizado un repaso general sobre las características con las que podemos encontrarnos en el entorno de ejecución de una aplicación **Android**, ¿qué bibliotecas adicionales hay que instalar para poder desarrollar con **Android**? ¿Es suficiente con el **SDK** clásico? ¿Es necesario o aconsejable tener también un **IDE**?

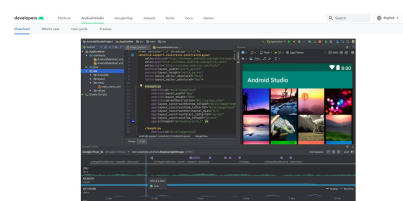
Para compilar un programa con **Android** se necesita como mínimo tener instalado el **SDK** el cual se puede descargar de por separado. A partir de ahí se pueden descargar e instalar el resto de herramientas (emuladores, entornos de desarrollo, depurador, etc.).

Del mismo modo que se haría con **Android**, podría hacerse con cualquier otra plataforma de desarrollo con la que se hubiera decidido trabajar: **Java ME, iOS, Windows Phone**, etc. En cada caso serían necesarias unas herramientas específicas (compiladores, editores, depuradores, emuladores, etc.). Todo esto lo hemos tratado ya en un apartado anterior.

En el caso particular de **Android** puedes optar por la posibilidad que se ha planteado de instalar el **SDK** junto con otros elementos o bien utilizar algún otro entorno compatible que traiga todas las herramientas necesarias incorporadas. Para este curso usaremos el entorno **Android Studio**, con los paquetes necesarios para trabajar con **Android**. Precisamente en la página oficial de **Android Studio** nos da la posibilidad de instalarnos el **IDE** Android Studio, incluyendo el **SDK**, que es la forma habitual y por defecto, o bien sin incluirlo.

Entorno integrado de desarrollo: Instalación de Android Studio

Para poder desarrollar una aplicación **Android** necesitarás un entorno con el que poder realizar tu trabajo habitual de programador (editores, compiladores, depuradores, emuladores, etc.). Dependiendo de tus preferencias personales y de las opciones disponibles tendrás que elegir entre las distintas alternativas que se te propongan. En nuestro caso se ha decidido que vamos a trabajar con el entorno **Android Studio**. Esta será la herramienta que utilizarás a partir de ahora y durante el resto del curso para el desarrollo de aplicaciones sobre dispositivos móviles.



[Google Developers](#) (License Apache 2.0)

Para la instalación de **Android Studio**, nos remitimos a la [página oficial de Android Studio](#)  .

En ella podrás visualizar un video descriptivo de todo el proceso de instalación sea cual sea tu sistema operativo, puesto que al ser una aplicación multiplataforma, está completamente testada y optimizada para los sistemas operativos más habituales (**Windows, Mac y Linux**)

En cuanto a los requisitos del sistema, necesitarás los especificados en la página oficial. Aunque será deseable disponer de los mejores recursos hardware de los que podamos disponer ya que el proceso de carga del emulador será algo tedioso por muy elevadas que sean las prestaciones del sistema.

Debes conocer

Es importante que conozcas los requisitos mínimos que Android Studio necesita para ejecutarse correctamente. Debes consultarlos en la parte inferior de la [página de descarga](#)  .

2.- Integración en el entorno de desarrollo.

Caso práctico

El siguiente paso es verificar que los ordenadores con los que van a trabajar cuentan con todos los requisitos necesarios para instalar el IDE **Android Studio** y descargarlo de la web oficial.

-Es importante usar fuentes oficiales de descarga para evitar problemas de seguridad en las aplicaciones que desarrollemos -indica Ada.

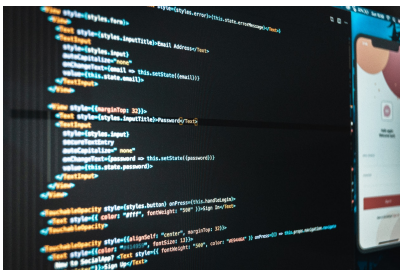
-Lo tendremos en cuenta -apuntan Juan, María y Pedro.

-Tras la descarga -continúa Ada-, si contamos con los permisos necesarios en el sistema, procederemos a su instalación para crear nuestro primer proyecto **Android**.

-¡Qué ganas! -responde el equipo al unísono.



[Mikhail](#) ([Pexels](#))



[Rodrigo Santos](#) ([Licencia de pexels](#))

Una vez se ha descargado e instalado en tu ordenador la aplicación **Android Studio**, debemos ver qué pasos debemos seguir para desarrollar, probar y ejecutar una mínima aplicación móvil con esta herramienta. A lo largo de los siguientes subapartados describiremos de manera concisa cómo proceder para crear la clásica aplicación *¡Hola mundo!* que marcará nuestro punto de partida para sumergirnos en el desarrollo de aplicaciones móviles para plataforma Android. ¡Comencemos!.

Recomendación

En caso de no contar con los permisos necesarios para instalar la aplicación en tu ordenador debes conseguirlos.

Podrías plantearte usar una máquina virtual cómo las que has utilizado en otros módulos del curso. Pero ten en cuenta que para poder probar tus aplicaciones, Android Studio usa lo que se llama un virtualizador de dispositivos Android (AVD). Este depende de la configuración de tu SW de virtualización (VirtualBox, HyperV,...), la configuración de tu BIOS e incluso tu microprocesador y hay muchos casos en los que no se comporta de forma adecuada.

Te recomendamos por tanto que uses una máquina real para evitar problemas con los emuladores y poder incluso instalar otros diferentes a los que te ofrece Android Studio de un modo más sencillo.

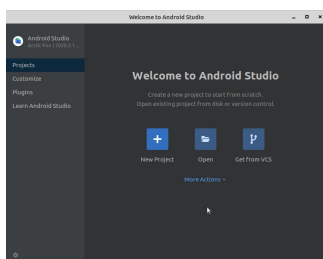
2.1.- Creación de un proyecto.

Para describir este apartado, usaremos la referencia proporcionada desde esta página de [Creación de proyectos de developer.android.com](https://developer.android.com/studio/projects/create-project)  .

Un proyecto de **Android Studio** contiene uno o más módulos que mantienen tu código organizado en unidades de funcionalidad discretas. Veremos la manera de iniciar un proyecto nuevo o importar uno existente.

Android Studio facilita la creación de apps de Android para varios factores de forma, como teléfonos y tablets, **Wear OS** , Android TV y **Android Automotive**. El asistente **New Project** te permite elegir los formatos para tu app y completar la estructura del proyecto con todo lo que necesitas para comenzar. Revisa y sigue los siguientes pasos para crear un proyecto nuevo.

Paso 1: Nuevo Proyecto



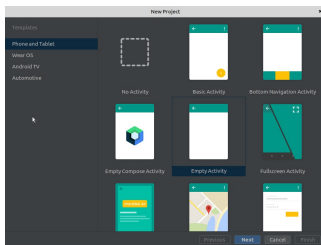
[Google Developers](https://developers.google.com/android) (Uso educativo nc)

Si no tienes un proyecto abierto, Android Studio te muestra la pantalla de bienvenida. Para crear un proyecto nuevo, haz clic en **New Project**.

Si tienes un proyecto abierto, Android Studio muestra el entorno de desarrollo. Para crear un proyecto nuevo, haz clic en **File > New > New Project**.

Paso 2: Agregar una actividad

En la siguiente pantalla, puedes seleccionar un tipo de proyecto. Escogeremos **Phone** and **Tablet** con una actividad vacía (**Empty Activity**)



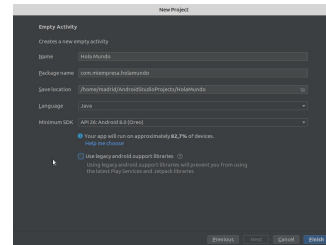
[Google Developers](https://developers.google.com/android) (Uso educativo nc)

Paso 3: Configuración del proyecto

Introduce los siguientes datos de configuración del nuevo proyecto que estás creando:

✓ Nombre de la aplicación: Hola mundo

- ✓ Nombre del paquete: com.miempresa.holamundo
- ✓ Ubicación para guardar el proyecto: por defecto o la que elijas
- ✓ Language: Java
- ✓ Nivel API mínimo: escoge aquel que permita que nuestra app se ejecute en valores cercanos al 100% de los dispositivos
- ✓ No marcar la opción Use Legacy android.support libraries



[Google Developers](#) (Uso educativo nc)

Pulsar el botón **Finalizar**.

A continuación debe aparecer el nuevo proyecto abierto en la ventana de Android Studio.

Paso 4: Seleccionar factores de forma y el nivel de API

Este paso no lo realizaremos ahora pero lo comentamos para verlo más adelante en el curso.

API Level	Device Support
1	0.0%
2	0.0%
3	0.0%
4	0.0%
5	0.0%
6	0.0%
7	0.0%
8	0.0%
9	0.0%
10	0.0%
11	0.0%
12	0.0%
13	0.0%
14	0.0%
15	0.0%
16	0.0%
17	0.0%
18	0.0%
19	0.0%
20	0.0%
21	0.0%
22	0.0%
23	0.0%
24	0.0%
25	0.0%
26	0.0%
27	0.0%
28	0.0%
29	50.8%
30	24.3%

[Google Developers](#) (Uso educativo nc)

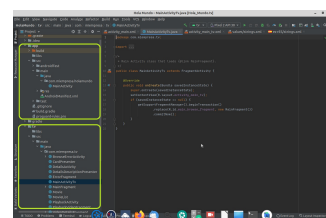
Podremos seleccionar los **factores de forma** admitidos por tu app (o añadirlos con **File > New > New Module**), como teléfonos y tablets, Wear OS , Android TV y Android Automotive. Los formatos seleccionados se convierten en los módulos de la app dentro del proyecto. Para cada formato, también puedes seleccionar el nivel de API para esa app. Asimismo, podremos seleccionar las versiones

mínimas para los distintos factores de forma. Esta selección hará que nuestra aplicación sea compatible con más o menos versiones de Android, tal y como se muestra en esta tabla.

Por ejemplo, según la imagen, si elegimos la API29, nuestra aplicación será compatible, en principio sólo con el 50,8% de todos los dispositivos Android tablet y teléfonos que hay en uso en el mundo.

NOTA: Para eliminar módulos añadidos, nos colocamos encima de la carpeta del módulo añadido y con el botón derecho "Open Module Settings" para eliminarlo con el "-". Después suprimimos la carpeta.

Cada factor de forma será un módulo en nuestra app, como se puede ver en la imagen de la derecha (en este caso hemos supuesto que hemos elegido los factores de forma Phone and Tablet y TV):



[Google Developers](#) (Uso educativo nc)

Paso 5: Actividad principal

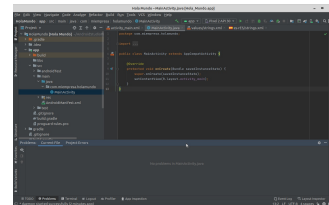
Una **actividad (activity)** es un componente de la aplicación que contiene una pantalla con la que los usuarios pueden interactuar para realizar una acción, como

marcar un número telefónico, tomar una foto, enviar un correo electrónico o ver un mapa. A cada actividad se le asigna una ventana en la que se puede dibujar su interfaz de usuario. La ventana generalmente abarca toda la pantalla, pero en ocasiones puede ser más pequeña que esta y quedar "flotando" encima de otras ventanas.

Una aplicación generalmente consiste en múltiples actividades vinculadas de forma flexible entre sí. Normalmente, una actividad en una aplicación se especifica como la actividad "principal" que se presenta al usuario cuando este inicia la aplicación por primera vez. Cada actividad puede a su vez iniciar otra actividad para poder realizar diferentes acciones.

Por ahora nuestro proyecto sólo tiene una actividad que es la principal representada por:

- A. La clase `MainActivity` (con fichero de código fuente `MainActivity.java`)
- B. La disposición o `layout activity_main` (almacenada en el fichero XML: `activity_main.xml`) que contiene la información necesaria para dibujar en pantalla la interfaz gráfica de la actividad principal



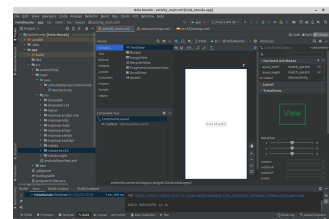
[Google Developers](#) (Uso educativo nc)

Nota: no es obligatorio, sin embargo, por ahora, la primera actividad que crearemos será la actividad principal de la aplicación. Mantendremos el nombre **MainActivity** con fichero de interfaz **activity_main**, lo cual ayudará a identificar y organizar las partes de la aplicación. Por otro lado, comentar que la actividad principal no tiene por qué ser la primera actividad que se abre en la aplicación, aunque será lo más frecuente. Puede suceder, que antes de abrir la actividad principal se abra lo que se conoce como una pantalla de bienvenida (*splash screen*) que ayuda a mejorar la experiencia de usuario. Esto será habitual en aplicaciones más trabajadas o en fase de producción.

Para ver la relación existente entre la clase **MainActivity** y su disposición (**layout**) podemos pulsar Ctrl+Blzq (Botón izquierdo del ratón) sobre el nombre de la clase MainActivity en la línea de código "`public class MainActivity`" y nos llevará al fichero XML asociado. Asimismo si pulsamos Ctrl+Blzq sobre el nombre de una clase, el entorno de desarrollo Android Studio nos llevará a su definición; puedes probar pulsando sobre *Bundle* (el tipo del parámetro del método onCreate). En el caso de MainActivity no nos llevó a su definición ya que estamos en ella (en el fichero `MainActivity.java` donde se define la propia clase); en vez de eso nos mostró los sitios donde se usa esta clase (uno de ellos es el fichero XML de la disposición asociada).

Paso 6: Desarrollar tu app

Android Studio crea la estructura predeterminada de tu proyecto y se abre el entorno de desarrollo. Si tu app admite más de un formato, Android Studio crea una carpeta de módulos con archivos de origen completos para cada uno de ellos, como se muestra en la siguiente figura, donde se puede apreciar que:



[Google Developers](#) (Uso educativo nc)

- ✓ En la parte superior nos encontramos con un menú y/o barra de herramientas.

- ✓ En el panel lateral izquierdo tenemos un navegador de la estructura de ficheros del proyecto.
- ✓ En parte central disponemos de un panel con una paleta de elementos incorporables en una emulación de la interfaz visual de mi aplicación.
- ✓ En el panel lateral derecho disponemos de un árbol de componentes de la interfaz visual, así como un descriptor de propiedades de cada elemento incorporado.

Para saber más

Para obtener más información sobre la estructura del proyecto y los tipos de módulos de Android, lee [Información general sobre proyectos](#)  .

Para obtener más información sobre la manera de agregar un módulo para un dispositivo nuevo a un proyecto existente, consulta [Agregar un módulo para un dispositivo nuevo](#)  .

2.2.- Escritura de código.

Lo más habitual, al menos en los primeros pasos con Android Studio es partir de una aplicación previa básica que a modo de esqueleto permita agregar las particularidades necesarias para dar respuesta a las necesidades a cubrir por nuestra aplicación. Es decir, aprovecharemos las plantillas que nos proporciona Android Studio, y sobre la que elijamos, comenzaremos a trabajar para desarrollar nuestra aplicación. Concretamente, vamos a trabajar sobre la plantilla "**Empty Activity**" que tiene los elementos estrictamente necesarios para poder ejecutar la aplicación generada. Además, es habitual también, al menos, en los primeros pasos con Android Studio, mantener el nombre de actividad que pone por defecto (**MainActivity**), de forma que cuando arranque la aplicación, esta será la actividad (**activity**) que cargará en primer lugar.

Posteriormente, podremos incorporar más actividades para completar nuestra aplicación. También podremos cambiar el nombre de cualquiera de ellas, incluido **MainActivity**. Todas las actividades que vayamos incorporando en nuestra aplicación quedarán reflejadas en el fichero **AndroidManifest.xml** que veremos posteriormente. Además dicho fichero permitirá establecer cuál será la actividad con la que arrancará la aplicación.

Según la documentación proporcionada por Android Developers:

Una Activity es un componente de la aplicación que contiene una pantalla con la que los usuarios pueden interactuar para realizar una acción, como marcar un número telefónico, tomar una foto, enviar un correo electrónico o ver un mapa. A cada actividad se le asigna una ventana en la que se puede dibujar su interfaz de usuario. La ventana generalmente abarca toda la pantalla, pero en ocasiones puede ser más pequeña que esta y quedar "flotando" encima de otras ventanas.

Una aplicación generalmente consiste en múltiples actividades vinculadas de forma flexible entre sí. Normalmente, una actividad en una aplicación se especifica como la actividad "principal" que se presenta al usuario cuando este inicia la aplicación por primera vez. Cada actividad puede a su vez iniciar otra actividad para poder realizar diferentes acciones. Cada vez que se inicia una actividad nueva, se detiene la actividad anterior, pero el sistema conserva la actividad en una pila (la "pila de actividades"). Cuando se inicia una actividad nueva, se la incluye en la pila de actividades y capta el foco del usuario. La pila de actividades cumple con el mecanismo de pila "el último en entrar es el primero en salir", por lo que, cuando el usuario termina de interactuar con la actividad actual y presiona el botón Atrás, se quita de la pila (y se destruye) y se reanuda la actividad anterior.

Así pues, de forma un poco simple, podremos asociar el concepto de **Activity** al de "pantalla" o ventana de la aplicación, de forma que todas las diferentes pantallas que pudiéramos ver en una aplicación tendrán asociadas una actividad.

Las actividades estarán compuestas por una parte lógica y una parte gráfica.

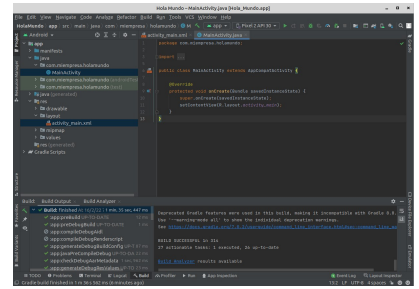
La parte lógica se corresponderá con un archivo **.java**, que contiene la clase para poder interactuar desde esa actividad, a través de los diferentes métodos de que disponga, e incluso apoyándose en otros ficheros **.java** que contengan clases que también interactúen entre sí.

Por otro lado, la parte gráfica normalmente se corresponderá con un fichero **.xml** en el que se define la disposición (**layout**) de los elementos que se mostrarán. Así, en dicho fichero, mediante etiquetas XML específicas, se podrán incorporar todos aquellos elementos que queramos que se muestren en mi interfaz.

Veremos más adelante, cuando planteemos alguna aplicación con interfaz "programática", que será posible incorporar los elementos gráficos directamente en el fichero **.java**, pero en general, esto suele ser bastante más costoso en esfuerzo y tiempo.

Resumiendo, la escritura se hará en ficheros **.java**, para incorporar el código (utilizando lenguaje de programación **Java**) responsable de la lógica de mi aplicación y en ficheros XML para incorporar el código (utilizando lenguaje de etiquetas XML) responsable de la presentación o aspecto de la interfaz.

Al crear nuestra primera aplicación, podemos ver ambas facetas. Para ello, completamos la creación de nuestro primer proyecto (llamado *HolaMundo*), y vemos que a través del navegador de proyectos ubicado en el marco (*frame*) de la izquierda se crean 2 ficheros relacionados entre sí:



[Google Developers](#) (Uso educativo nc)

- ✔ MainActivity, es la clase Java definida en el fichero *MainActivity.java*, y que contendrá las especificaciones lógicas de la actividad principal y de momento única, de nuestra aplicación.
- ✔ *activity_main.xml* contendrá las especificaciones de apariencia o diseño de la interfaz de esa actividad.

En los siguientes subapartados veremos el contenido de cada fichero y los explicaremos con más detalle.

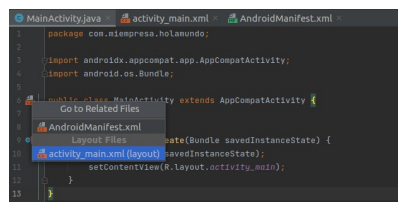
2.2.1.- MainActivity.

Ejercicio Resuelto. Contenido de MainActivity.

En el capítulo anterior se explicó que se iba a utilizar el lenguaje Java, el IDE Android Studio y el formato XML.

1. Averigua los archivos XML relacionados con la clase java principal de la aplicación.
2. Haz una breve explicación de cada una de las líneas de código java del archivo MainActivity.java

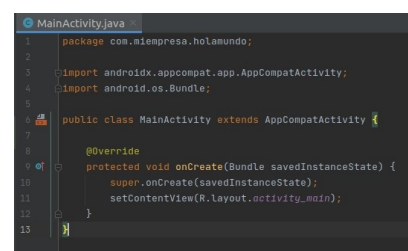
1. Debes pulsar en el icono "Related XML file" tal y cómo se muestra en la imagen para ver los ficheros XML que están relacionados con la clase MainActivity: *activity_main.xml* y *AndroidManifest.xml*.



[Google Developers](#) (Uso educativo nc)

2. Breve explicación del código:

- ✓ En la línea 1 se indica el nombre del paquete en donde se incluye nuestra clase.
- ✓ Con las líneas 3 y 4 incorporamos clases necesarias para nuestra app, las cuales están en otros paquetes.
- ✓ En la línea 6 definimos la clase llamada "MainActivity" y le aplicamos el mecanismo de herencia habitual propio de Java haciendo que extienda de la clase AppCompatActivity. Esta clase que hemos importado anteriormente dispone de las definiciones necesarias para cargar una actividad.
- ✓ En la línea 9 se incorpora una sobreescripción del método "onCreate". **Todas las actividades (activities) deben llevar el método "onCreate"**, el cual hará que inicie la actividad. Se debe pasar como parámetro un objeto de la clase *Bundle*, el cual permitirá inicializar el estado de la actividad. Si no programamos este método la actividad no se iniciará.
 - ➡ Lo primero que tendremos que hacer en este método será llamar al método onCreate de la clase padre pasando el objeto Bundle anterior.



[Google Developers](#) (Uso educativo nc)

- ➡ A continuación, a través de la línea de código: `setContentView(R.Layout.activity_main)` conseguiremos enlazar la parte lógica con la parte gráfica. El archivo XML que va a mostrarse cuando se ejecute la clase "MainActivity" será el archivo XML llamado dentro del registro de recursos de Android Studio llamado "activity_main".

En general, cada vez que creamos una actividad nueva con un determinado nombre, debo hacer que herede de la clase `AppCompatActivity` (o incluso de una manera más general, de la clase `Activity`) y además debe tener vinculado un archivo XML que relacionaré a través del método "`setContentView`", pasando como parámetro el nombre que recibe dentro del registro de recursos que tiene Android y que para las actividades tiene prefijo "`R.layout.`", como por ejemplo sucede en el caso anterior: `R.layout.activity_main`.

Debes conocer

En la línea 3, si pulsas `Ctrl+Alt+Q` sobre `AppCompatActivity` se abre el fichero de código fuente donde se define el paquete en el que se incluye dicha clase. Idem para la clase `Bundle` de la sentencia `import` de la línea 4. Observa brevemente la descripción de esas clases en el comentario (`/** ... */`) que aparece encima de la definición de la clase.

2.2.2.- activity_main.xml.

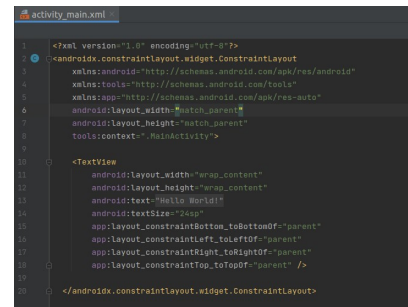
Ejercicio Resuelto

Visualiza el contenido del archivo *activity_main.xml* que define cómo se mostrará el contenido de tu aplicación e intenta entender su contenido.

En la imagen se muestra el contenido del archivo *activity_main.xml*.

Pasemos a explicar qué significa este código XML:

- ✓ En la línea 1, ese primer elemento del fichero XML define la versión de XML y la codificación de caracteres utilizada. Habitualmente mantendremos estos valores.
- ✓ En la línea 2, la etiqueta `<android.support.constraint.ConstraintLayout...>` determina el tipo de disposición (**layout**) que queremos que se establezca, e incorpora una serie de atributos correspondientes a los espacios de nombres indicando unas URI's para identificarlos. Además tiene otros atributos que caracterizarán la anchura y altura de la disposición (**layout**) y una especificación del contexto de esa disposición (**layout**) que servirá para recordar que actividad (**activity**) la asociará.
- ✓ Por otro lado tenemos una etiqueta `<TextView...>` que corresponde a una etiqueta de tipo texto que tendrá comportamiento similar al control *Label* de Java o *label* de html. Dispondrá de atributos para especificar anchura y altura, así como el texto contenido y además una serie de atributos correspondientes a la forma en que se situará este control dentro de la disposición (**layout**).



```
<?xml version="1.0" encoding="utf-8"?>
<android.support.constraint.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        android:textSize="24sp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</android.support.constraint.ConstraintLayout>
```

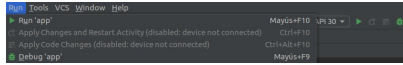
[Google Developers](#) (Uso educativo no)

Para saber más

Para saber más sobre el acceso a recursos de Android Studio, será muy interesante el siguiente enlace: [Acceso a recursos en Android Studio](#) 

2.3.- Compilación.

Para compilar el proyecto basta con ir a **Build > Make Project** y veremos que nos notifica que empezará a ejecutarse Gradle.



[Google Developers](#) (Uso educativo no)

Para compilar y además ejecutar tu app, bastará con hacer clic en el botón **Run** situado en la barra de herramientas, o bien, a través de menú **Run > Run 'app'**.

Android Studio compilará la app con **Gradle**, y a continuación, solicitará que seleccionemos un destino de implementación (un emulador o un dispositivo conectado) y, luego, cargará la app en dicho destino. Si no tenemos ningún dispositivo preparado para poder emular la aplicación, Android Studio mostrará un mensaje similar a este: *Error running app: No target device found*.

Podremos personalizar parte de este comportamiento predeterminado (por ejemplo, la selección de un destino de implementación automático). Veremos en siguientes apartados aspectos relacionados con el emulador y la ejecución de aplicación sobre él y otros aspectos relacionados con la ejecución en un dispositivo físico.

Debes conocer

Además, puedes ejecutar tu app en el modo de depuración. Para ello, haz clic en el botón de **Debug**. La ejecución de tu app en el modo de depuración te permite configurar puntos de interrupción en el código, examinar variables y evaluar expresiones en el tiempo de ejecución, y también ejecutar herramientas de depuración.

Para obtener más información, consulta [Depuración de tu app](#)  .

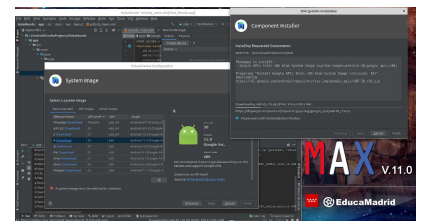
2.4.- Emuladores.

Normalmente los entornos de desarrollo integrados suelen incorporar uno o varios emuladores que te servirán para poder probar las aplicaciones antes de instalarlas en el dispositivo para el cual se han programado. Así podrás observar si la aplicación funciona correctamente y si tiene más o menos el aspecto que tenías pensado. Hay que tener en cuenta que los emuladores no siempre van coincidir en aspecto (y a veces incluso en funcionalidad) con el dispositivo real y podrás encontrar abundantes diferencias dependiendo de diversos factores como por ejemplo el tipo de dispositivo que se emule. No es lo mismo emular un teléfono con **Windows Phone**, que un **iPhone** o un **iPad**, o un teléfono con **Android**, o un dispositivo móvil con **Java ME**.

Independientemente de la plataforma para la que vayas a desarrollar las aplicaciones (**Android**, **iOS**, **Java ME**, etc.) es importante que el entorno que elijas disponga de emuladores pues es lo que utilizarás la mayor parte del tiempo. Gran parte del proceso de depuración se realizará con el emulador, podremos ejecutar las instrucciones paso a paso, establecer puntos de ruptura (breakpoints), observar el contenido de variables, etc. Además en muchos casos es posible que no tengas a tu disposición en ese momento el dispositivo concreto para el que deseas programar de manera que necesitarás el emulador.

En principio trataremos de trabajar con la aplicación de emulación proporcionada por Android (**AVD**), pero es posible utilizar otras aplicaciones gratuitas o de pago que consiguen emulaciones tan buenas o incluso más rápidas que la AVD. Éstas, normalmente se valen del uso de máquinas virtuales para poder funcionar.

Hay que tener en cuenta que si decidimos cambiar el tipo de emulador utilizado, la salida podrá ser ligeramente diferente en cuanto al aspecto estético del dispositivo emulado.



[Google Developers](#) (Uso educativo nc)

Si deseamos usar [Android Emulator](#)  para ejecutar nuestra app, debes preparar un **Android Virtual Device (AVD)**. Si todavía no lo hemos creado, cuando hagamos clic en Run, aparecerá un error ya que la app no encuentra un dispositivo sobre el que ejecutarse. Para solventar el problema debemos abrir **Tools > Device Manager** desde donde podremos elegir **Create Device**. Seguiremos las instrucciones del asistente **Virtual Device Configuration** para definir el tipo de dispositivo que deseas emular, seleccionando la categoría (teléfono, TV, Wear, tablet o Automotive), el modelo y la imagen del sistema (a descargar).

Para saber más

Para obtener más información, consulta [Creación y administración de dispositivos virtuales](#) .

También puedes conseguir más información sobre otros [emuladores de Android](#) , algunos orientados sólo para los juegos, pero otros, como **Genymotion** orientados al desarrollo de apps.



2.5.- Instalación y ejecución en un dispositivo real.

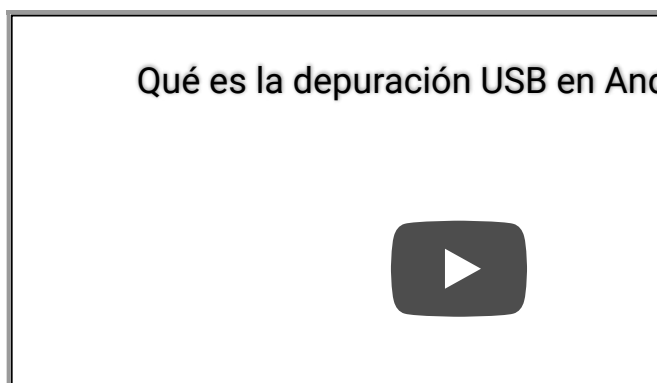
Como ya se ha comentado antes, algo que debes tener en cuenta cuando se desarrollan aplicaciones con la tecnología Android Studio (y en general para casi cualquier plataforma) es que el aspecto e incluso el funcionamiento que presente la aplicación puede que sea diferente en un dispositivo real con respecto al que se podía observar en un emulador. Es muy posible que su aspecto varíe según se instale en un dispositivo u otro, así como también si lo pruebas en un emulador u otro.

Esto se debe tanto a las características físicas del propio dispositivo (tamaño, resolución y colores de la pantalla, tipo de mando de juegos (joystick), botones, etc.) como a la implementación de la máquina virtual de Java que tenga ese dispositivo. Aunque la comparación no es del todo precisa, la situación podría ser similar a la que se produce cuando navegas por un mismo sitio web con ordenadores distintos, con sistemas operativos diferentes y con navegadores también distintos, obteniendo visualizaciones de la web algo diferentes.

La manera más sencilla de probar tu aplicación será transfiriendo el archivo apk generado de la aplicación a la memoria del dispositivo donde deseas realizar la prueba (a través de un cable USB, por Bluetooth, mediante una tarjeta de memoria, etc.).

Dependiendo del fabricante del dispositivo, es posible que se disponga de alguna utilidad específica del fabricante que permita realizar la instalación desde el ordenador pero también podemos disponer de utilidades no propietarias de fabricantes de móviles. Un ejemplo de este SW para Android es adb. La forma de hacerlo podría cambiar dependiendo del fabricante del dispositivo así como del tipo de archivos generados (no será el mismo proceso para un iOS que para un Android, cada plataforma tendrá sus procedimientos específicos de instalación de software).

En el caso de Android, será imprescindible activar la opción de **Depuración por USB**. Puedes ver más información al respecto en este vídeo:



[El Androide Libre.](#) (Todos los derechos reservados)

[Descripción textual del vídeo](#) 

Para saber más

Para obtener más información, consulta [Ejecutar apps en un dispositivo de hardware](#)  .

3.- Gestión del entorno de ejecución.

Caso práctico

El equipo de desarrollo está entusiasmado comprobando las opciones que Android Studio proporciona para programar y testear con dispositivos virtuales e incluso reales.

-Tenemos muchas ganas de ponernos a escribir código de funciones -explica Pedro a Ada-, y empezar a diseñar el aspecto de las aplicaciones.

-Tranquilos, todo llegará -responde Ada-. Antes debéis seguir aprendiendo algunos detalles importantes relacionados con el contexto de una aplicación en un dispositivo móvil.

-¿Cuál es el siguiente paso? -pregunta María.

-Quiero estar segura de que todos conocéis el administrador de aplicaciones de Android -explica Ada-, y que sois capaces de instalar y desinstalar aplicaciones. No es complicado pero debo mostraros algunas peculiaridades en cuanto a las **fases** por las que una aplicación pasa.

-Sin embargo -prosigue Ada-, existen otros conceptos mucho más complejos relacionados con la Ingeniería del desarrollo del Software y es imprescindible saber que las aplicaciones pueden pasar por distintos **estados** mientras están en ejecución.

-Estamos preparados para aprender los conceptos de fases y estados -asegura Juan.

Gracias a algunos diagramas de la página web de Android Developers, Ada explica al equipo estos conceptos así como los eventos que lanzarán los cambios de los estados en las aplicaciones de Android.



[Mikhail \(pexels\)](#)

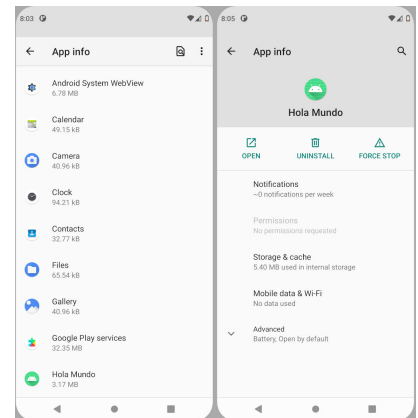
Para describir el entorno de ejecución en un sistema como Android debemos considerar 2 aspectos importantes:

- ✓ El ciclo de vida de una aplicación. Fases y administrador de aplicaciones.
- ✓ El modelo de estados de una aplicación en ejecución.

3.1.- El ciclo de vida y el administrador de aplicaciones.

El administrador de aplicaciones o gestor de aplicaciones permite mostrar todas las apps que tenemos instaladas en nuestro dispositivo y controlar de un vistazo las que se está ejecutando, desinstalarlas o detenerlas, o incluso conocer en cada momento los recursos que está consumiendo cada una de ellas. Llevará a cabo esencialmente dos funciones:

- ✓ La gestión del ciclo de vida de las aplicaciones en el dispositivo.
- ✓ El control de los estados por los que pasa la aplicación mientras resida en la memoria del dispositivo, es decir, mientras esté en ejecución.



[Google Developers](#) (Uso educativo nc)

El ciclo de vida de una aplicación de Android

El ciclo de vida de una aplicación de Android pasa por cinco fases: descubrimiento, instalación, ejecución, actualización y borrado.

- ✓ **Descubrimiento (o localización).** Consiste en el proceso por el cual un usuario localiza una aplicación Android a través de su dispositivo. El gestor de aplicaciones es el que proporciona los mecanismos necesarios para realizar la elección de la aplicación que se desea descargar (mediante un cable USB, Bluetooth, conexión a Internet, etc.).
- ✓ **Instalación.** Una vez que la aplicación ha sido descargada en el dispositivo, ésta puede ser instalada. El gestor de aplicaciones de Android se encargará de controlar el proceso de instalación informando al usuario de la evolución de este proceso. Una vez instalado la aplicación, éste permanecerá en el dispositivo (en memoria persistente) hasta que el usuario decida borrarlo.
- ✓ **Ejecución.** El gestor de aplicaciones de Android es también el que permite poner en ejecución la aplicación. Una vez que una aplicación está en ejecución, el gestor de aplicaciones también tendrá el control de los distintos estados por los que pueda pasar dicha aplicación dependiendo de los eventos que se vayan desencadenando durante esa ejecución.
- ✓ **Actualización.** Una vez que una aplicación ha sido descargada, el gestor de aplicaciones de Android debe ser capaz de detectar si se trata de una actualización de una aplicación ya presente en el dispositivo. En tal caso se informaría al usuario para que decidiera si quiere llevar a cabo o no la actualización.
- ✓ **Borrado.** Si el usuario así lo desea, el gestor de aplicaciones de Android se encargará de borrar la aplicación del dispositivo. Normalmente se solicitará confirmación y se informará de las posibles incidencias durante el proceso de borrado.

3.2.- Modelo de estados de una aplicación.

De manera similar a como el planificador de procesos (scheduler) en sistemas operativos se encarga de gestionar el estado de los procesos (programas en ejecución), el Administrador de Aplicaciones es también el encargado de gestionar los estados de una aplicación durante su ejecución.

Cuando una aplicación se pone en ejecución es cargada en la memoria del dispositivo mediante un **proceso, entrando en funcionamiento una activity**. Es una diferencia con respecto a otros paradigmas que cargan la aplicación mediante el método **main()**. El ciclo de vida de cualquier activity dependerá de cómo esté relacionado con otras actividades, con sus tareas y con la pila de actividades que tenga la aplicación.

Según [Android Developer](#) , los usuarios cambian el estado de los procesos de las apps según interactúan con ellas. Los cambios en el estado del proceso van a cambiar el estado de la activity que se está ejecutando. Básicamente estarán en tres estados:

- ✓ **Reanudada:** La actividad se encuentra en el primer plano de la pantalla y tiene la atención del usuario. (A veces, este estado también se denomina "en ejecución").
- ✓ **Pausada:** Otra actividad se encuentra en el primer plano y tiene la atención del usuario, pero esta todavía está visible. Es decir, otra actividad está visible por encima de esta y esa actividad es parcialmente transparente o no cubre toda la pantalla. Una actividad pausada está completamente "viva" (el objeto Activity se conserva en la memoria, mantiene toda la información de estado y miembro y continúa anexado al administrador de ventanas), pero el sistema puede eliminarla en situaciones en que la memoria sea extremadamente baja.
- ✓ **Detenida:** La actividad está completamente opacada por otra actividad (ahora la actividad se encuentra en "segundo plano"). Una actividad detenida también permanece "viva" (el objeto [Activity](#)  se conserva en memoria, mantiene toda la información de estado y miembro, pero no está anexado al administrador de ventanas). Sin embargo, ya no está visible para el usuario y el sistema puede eliminarla cuando necesite memoria en alguna otra parte.

Si se pausa o se detiene una actividad, el sistema puede excluirla de la memoria al solicitarle que se cierre (llamando a su método **finish()**), o simplemente eliminando su proceso. Cuando se vuelve a abrir la actividad (después de haber sido eliminada o destruida), es necesario crearla nuevamente.

Cuando una actividad entra y sale de los diferentes estados que se describieron más arriba, esto se notifica a través de diferentes métodos llamados **callback**. Todos los métodos callback son enlaces que puedes invalidar para realizar las tareas correspondientes cuando cambia el estado de la actividad. La siguiente actividad de ejemplo incluye todos los métodos fundamentales del ciclo de vida.

NOTA: La implementación que realices de estos métodos del ciclo de vida siempre debe llamar a la implementación de la superclase antes de realizar cualquier otra tarea, como se muestra en los ejemplos.

Ejemplo

```
1 public class ExampleActivity extends Activity {
2
3     @Override
4
5     public void onCreate(Bundle savedInstanceState) {
6
7         super.onCreate(savedInstanceState);
8
9         // The activity is being created.
10
11     }
12
13     @Override
14
15     protected void onStart() {
16
17         super.onStart();
18
19         // The activity is about to become visible.
20
21     }
22
23     @Override
24
25     protected void onResume() {
26
27         super.onResume();
28
29         // The activity has become visible (it is now "resumed").
30
31     }
32
33     @Override
34
35     protected void onPause() {
36
37         super.onPause();
38
39         // Another activity is taking focus (this activity is about to be "pausec
40
41     }
42
43     @Override
44
45     protected void onStop() {
46
47         super.onStop();
48
49         // The activity is no longer visible (it is now "stopped")
50
51     }
52
53     @Override
54
55     protected void onDestroy() {
56
57         super.onDestroy();
```

```

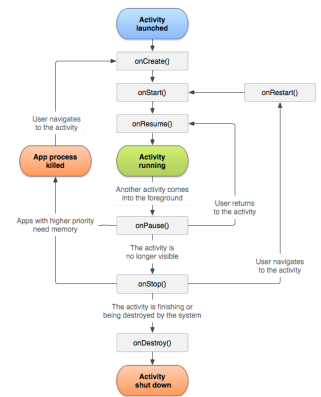
58
59         // The activity is about to be destroyed.
60
61     }
62
63 }

```

Los 3 bucles del ciclo de vida de un activity

En conjunto, estos métodos anteriores definen el ciclo de vida completo de una actividad. Al implementar estos métodos, puedes monitorear los tres bucles anidados en el ciclo de vida de la actividad que puedes ver representados en la ilustración así como el camino que debe tomar una actividad entre estados. Los rectángulos representan los métodos callback que puedes implementar para realizar operaciones cuando la actividad cambie de estado.

- ✓ El ciclo de vida **completo** de una actividad transcurre entre la llamada a **onCreate()** y la llamada a **onDestroy()**. Tu actividad debe configurar el estado "global" (como la definición del diseño) en **onCreate()**, y liberar todos los recursos restantes en **onDestroy()**. Por ejemplo, si hay un subproceso de descarga de datos de la red en ejecución en segundo plano en tu actividad, esta podría crear ese subproceso en **onCreate()** y luego detenerlo en **onDestroy()**.
- ✓ El ciclo de vida **visible** de una actividad transcurre entre la llamada a **onStart()** y la llamada a **onStop()**. Durante ese tiempo, el usuario puede ver la actividad en pantalla e interactuar con ella. Por ejemplo, se llama a **onStop()** cuando se inicia una nueva actividad y esta ya no está visible. Entre estos dos métodos, puedes conservar los recursos necesarios para mostrarle la actividad al usuario más tarde. El sistema podría llamar a **onStart()** y **onStop()** muchas veces durante el ciclo de vida completo de la actividad, ya que la actividad pasa de ser visible y a estar oculta para el usuario. No uses estos métodos para hacer acciones que sólo se realizar una vez.
- ✓ El ciclo de vida **en primer plano** de una actividad transcurre entre la llamada a **onResume()** y la llamada a **onPause()**. Durante ese tiempo, la actividad se encuentra al frente de todas las demás actividades en la pantalla y tiene el foco en la interacción del usuario. Con frecuencia, una actividad puede entrar y salir de primer plano, por ejemplo, se llama a **onPause()** cuando el dispositivo entra en suspensión o cuando aparece un diálogo. Dado que este estado puede cambiar con frecuencia, el código en estos métodos debe ser bastante liviano para evitar las transiciones lentas que hacen que el usuario deba esperar.



[Google Developers](#) (Uso educativo nc)